Computers & Security 154 (2025) 104413

Contents lists available at ScienceDirect

Computers & Security

journal homepage: www.elsevier.com/locate/cose

Full length article

LinTracer: An efficient tracking system for cyberattack chains fusing entity and event semantics

Tiantian Zhu [a], Wenya He [a], Tieming Chen [a], Jiabo Zhang [a], Mingqi Lv [a], Hongmei Li [a], Aohan Zheng [a], Jie Zheng [a], Mingjun Ma [a], Xiangyang Zheng [b], Zhengqiu Weng [b,*], Shuying Wu [c,*]

[a] Zhejiang University of Technology, China
[b] Wenzhou University of Technology, China
[c] Wenzhou Polytechnic, China

ARTICLE INFO

Keywords:
Cyber threat detection
Provenance graph
Attack investigation
Advanced persistent threat
Attack chain tracking

ABSTRACT

With the rapid development of information technology, advanced persistent threat (APT) attacks are becoming increasingly prevalent. This form of attack is known for its persistence, diversity, and stealth, and it results in serious security threats and economic losses for various organizations and institutions. In the face of this threat, tracing the attack chain (i.e., attack investigation) is critical to understanding the attacker's behavior, identifying attack methods and patterns, and taking appropriate defensive measures. However, the current APT attack investigation techniques suffer from insufficient audit log refinement, attack entrance location difficulties, and attack path tracking accuracy challenges. In this paper, we propose LinTracer, which is an efficient attack investigation system based on the ATT&CK attack model for Linux systems that fuses entity and event semantics for cyber-attack chains. First, an auditing mechanism is used to stably collect the kernel data of the target operating system, and data compression techniques are used to refine the log data and reduce the overhead imposed by the attack investigation system. Second, a backward causal analysis is performed from the alarm point to construct a suspicious provenance graph. LinTracer extracts the features used to distinguish between attack events and benign events, calculates the feature scores of the events, and then uses the backward propagation algorithm to propagate the dependency scores backward from the alarm point to identify the attack entry points. Finally, entity semantic labels are designed based on the ATT&CK framework to perform forward label propagation on the attack entry points, ultimately enabling an effective attack investigation. The experimental results derived from laboratory tests and DARPA Engagement (approximately 64 million auditing events obtained from real systems) show that LinTracer has good real-time performance and can accurately identify attack chains.

1. Introduction

Considering the continuous evolution of cyber threats, APT attacks pose a serious challenge to network security. APT attacks usually exploit multiple types of vulnerabilities and penetrate the target system in phases, resulting in serious financial losses for enterprises. To minimize the impact of attacks and prevent similar attacks, several teams have investigated APT attacks through provenance graphs. Provenance graphs depict the relationships (e.g., read, write, and clone) between system subjects (e.g., processes) and system objects (e.g., files, sockets, and pipes). The main methods used for conducting attack investigations include anomaly score-based methods (Liu et al., 2018; Hassan et al., 2020b, 2019; Fang et al., 2022), label delivery-based methods (Milajerdi et al., 2019; Hassan et al., 2020a; Xiong et al., 2020; Hossain et al., 2017; Zhu et al., 2023), and machine learning-based methods (Alsaheel et al., 2021; Xu et al., 2022; Zeng et al., 2021; Li et al., 2021; Zhao et al., 2020; Wang et al., 2020; Ding et al., 2023), as shown in Table 1. Anomaly score-based methods usually combine alert point features and anomaly score diffusion algorithms to quantify the suspiciousness of each edge between pairs of nodes to discover the most suspicious event behaviors in the target system, are able to capture new types of attacks that are difficult to cover with traditional rules, with high real-time performance, and are able to quickly respond to and identify anomalous behaviors. However, owing to their dependence on abnormal behaviors, these approaches may generate false alarms for some normal but atypical behaviors, resulting in a high false positive rate. Labeling-based delivery methods track and trace network attack paths

* Corresponding authors.
E-mail addresses: derisweng@wzut.edu.cn (Z. Weng), wushuying@wzpt.edu.cn (S. Wu).

https://doi.org/10.1016/j.cose.2025.104413
Received 16 July 2024; Received in revised form 5 February 2025; Accepted 2 March 2025
Available online 11 March 2025
0167-4048/© 2025 Elsevier Ltd. All rights are reserved, including those for text and data mining, AI training, and similar technologies.

by tagging data streams, hosts, or other key network information and delivering these labels to connected network nodes. However, since the persistence of APT attacks is usually accompanied by a large amount of log storage, the use of tag passing may lead to the label contamination problem, which can trigger errors in the results. Machine learning-based approaches enable the detection of anomalous behaviors and potential attack sources by training on a large amount of network traffic, log data, or other security events so that the developed machine learning model is able to learn and identify patterns that differ from normal behaviors. However, this type of method is highly dependent on a large amount of labeled training data, and the constructed model may perform poorly when faced with specific scenarios that lack sufficient labeling; at the same time, this strategy cannot support the training of traceability models with high generalization capabilities when faced with complex and diverse APT attacks.

Challenges. The current research on APT attack investigations faces a series of challenges, such as insufficient audit log conciseness, difficulties in accurately localizing attack entry points, and inaccuracy reconstructed attack paths, which make satisfying the demand for conciseness, precision, and comprehensibility in real scenarios difficult. Specifically, the shortcomings and main challenges of the existing research are as follows.

First, a lack of conciseness is observed. The current APT attackers often use diverse technical means to conduct attacks through complex and clever attack paths, including zero-day vulnerability exploitation, social engineering attacks, etc. Therefore, APT attacks therefore often persist for some time, which in turn leads to the storage of many logs. These logs cover a variety of events and information, which may have complex correlations. When conducting an attack investigation, these large number of logs and complex interconnections lead to dependency graph inflation and dependency explosion (Liu et al., 2018; Hassan et al., 2020b; King and Chen, 2003; King et al., 2005), which ultimately means that the investigation results will contain a large amount of redundant information; this information is difficult to simplify and has intricate and complex features. To solve the dependency explosion problem and the complexity of calculating a large amount of data, many previous studies (Xiong et al., 2020; Lee et al., 2013; Ma et al., 2015) have focused on data compression. However, many methods overcompress the input data during this process and may suffer from large semantic losses. Among such methods, NODOZE (Hassan et al., 2019) determines the level of suspicion on the basis of rules and filters some low-suspicion (Ma et al., 2016) process log data by filtering certain system events or files. Ma et al. (2015) proposed a processing method that overly relies on the level of development and fails to guarantee the integrity of logs. All these methods may lead to the loss of attack events and the loss of forward/backward reachability, making it difficult to restore the complete attack path and hindering the attack investigation process. In addition, numerous approaches implement attack investigations with the help of provenance graphs; when faced with many logs, they are not able to trace back a clear and concise attack path. For example, DEPIMPACT (Fang et al., 2022) overlaps the forward traceability graph with the backward traceability graph to obtain the final result, which is a highly complicated procedure.

Second, entry points are difficult to locate. Owing to the covert nature of APT attacks, when tracing the attack paths associated with indicator-of-compromise (IOC) or point-of-interest (POI) events, a multitude of potentially false attack entry points are often confronted. This situation not only significantly increases the workload imposed on the analyst but also leads to time-consuming and tedious investigations and alert fatigue. In some cases where causal analysis methods (Kwon et al., 2018, 2016; Ma et al., 2016; Zhu et al., 2023) are used, while these methods can increase the precision of causal relationships or the accuracy of constructed dependency graphs, the resulting dependency graphs may still contain multiple potential attack entry points. These entrances may be the actual causes of POI events, but they may also have been misclassified by algorithms or rules. Failing to further screen and differentiate these attack entrances will cause unnecessary difficulties for the subsequent attack investigations. Therefore, a lack of precision may lead to the introduction of more uncertainty in the analysis process, and this is a core problem that needs to be solved in the current attack investigation research.

Third, the obtained attack paths may be inaccurate. During APT attack research and judgment processes, accurately identifying and tracing the target attack path is crucial for effectively defending against and responding to the attack. Reverse tracing from the alarm point to the entry point via backward causal analysis (King and Chen, 2003; King et al., 2005; Goel et al., 2005) may generate multiple potential attack paths. Since APT attacks typically involve multiple stages and complex operations, this can lead to many paths for the characterization process, making it difficult to determine which of them actually represent the attacker's path of action.

Contributions. In this paper, we propose an efficient attack investigation system based on the ATT&CK attack model for Linux systems that fuses entity and event semantics for network attack chains. The system identifies suspicious attack entrances via an anomaly propagation algorithm by constructing a suspicious event graph from massive Linux system logs, extracting the features of the events and calculating the anomaly scores of the relevant events. The attack entrances are subsequently analyzed in the forward direction by specifying the labels and their scores, events, and other rules. On this basis, label combinations and path features are extracted, and the path score algorithm is applied in retrospective research and attack judgment work, which is an efficient and reliable means of tracking and analyzing the network security of the examined Linux system. Overall, the contributions of this paper include the following:

(1) We utilize redundant semantic skipping and temporary file deletion to solve the dependency explosion problem, and we greatly improve the efficiency of attack investigations while reducing the imposed data storage and operation overhead, making the final traceability research and judgment results more refined. Unlike the data compression methods used in previous studies, the log data compression method proposed in this paper has a better compression effect and does not negatively affect the subsequent APT attack investigation.

(2) We propose an event semantics-based APT attack tracing method in which feature values are computed by analyzing the global and local features of each event. Then, the feature scores of each edge are computed via the K-SUMS clustering algorithm and the discriminative feature projection scheme, which is based on linear discriminant analysis (LDA). Finally, the feature scores of the edges are propagated to the nodes via the entity semantic score propagation algorithm to determine the suspicious attack entrances and their paths. This approach not only overcomes the problem of attack bypass, which may be caused by local features, but also addresses the issue that global features may have low scores for certain critical events; it is also able to efficiently trace the potential entry points of multiple attacks.

(3) We propose an APT attack research and judgment method based on tag entity semantics, which performs forward analysis of the attack path while tagging and transferring tags and then further studies and determines the attack path by extracting tag combinations and event features, using path-level tag semantics computation methods, which improves the accuracy of the attack path. Moreover, through this method, the comprehensive nature of the attack path is further emphasized, which helps in understanding the behavior and strategy of the attacker in detail and increases the depth and precision of attack chain tracking.

(4) We integrate the proposed key techniques and present an efficient cyber-attack chains tracking system that incorporates entity semantics. Through experiments conducted on real simulated APT attacks and publicly available datasets, the system is shown to perform well and is able to efficiently track the attack entrances of multiple

Table 1
Comparison among the related works on attack investigation performance. Column 6 indicates the integrity of the data obtained after compression. The solidness of each marked circle reflects the corresponding degree: High ●, Medium ◑, Low ○

Methodology	System	False positive rate	False negative rate	Timeliness	Integrity
Label delivery-based	SLEUTH (Hossain et al., 2017)	○	◑	●	–
	APTSHIELD (Zhu et al., 2023)	◑	●	●	√
Anomaly score-based	NODOZE (Hassan et al., 2019)	●	○	◑	–
	DEPIMPECT Fang et al. (2022)	◑	○	○	×
Machine learning-based	ATLAS (Alsaheel et al., 2021)	○	◑	◑	×
	DEPCOMM (Xu et al., 2022)	◑	●	◑	×
	AIRTAG (Ding et al., 2023)	●	◑	◑	×
–	Lintracer	○	○	●	√

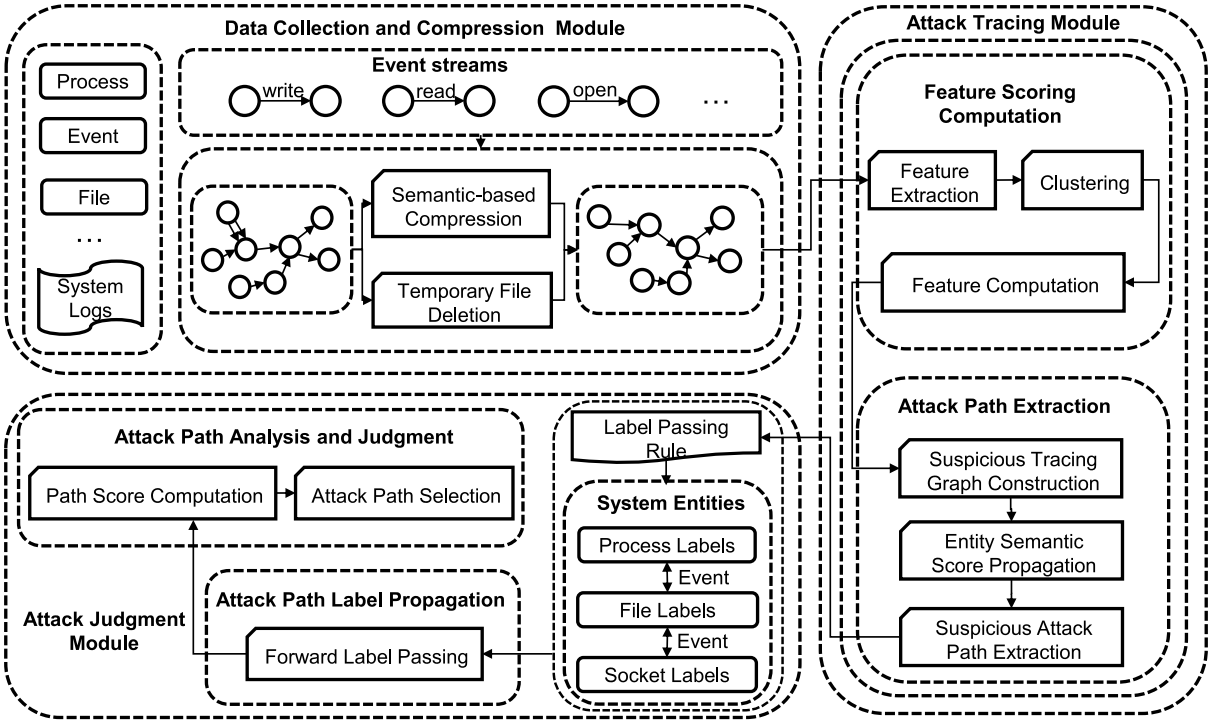

Fig. 1. Architecture of LinTracer.

types of attacks with more concise attack paths while achieving a low false alarm rate; it outperforms some of the existing state-of-the-art methods. We evaluate the proposed method on both the DARPA dataset and a lab dataset. The evaluation results show that our approach is more effective at revealing critical edges and attack entry points than DEPIMPACT (Fang et al., 2022).

2. Background knowledge

2.1. System audits

System auditing (Liu et al., 2018; Kwon et al., 2018; Ma et al., 2018) is an important security analysis method that collects audit logs of system calls and describes the interactions between system entities. With the continuous development of the major operating systems (e.g., Windows, Linux, and Mac OS), system auditing has become a necessary tool for addressing diverse threats (Jiang et al., 2006; Gao et al., 2018b,a, 2021; Jiang et al., 2006). In this work, system entities are involved mainly in files, processes, and sockets, and the interactions among these entities are mapped to three main categories of system events through system calls: file access, process creation and destruction, and network access events. System events are abstracted as interactions between subjects, operations, and objects; these interactions are represented as triples (Xu et al., 2022), where subjects are usually represented as processes originating from software applications and objects can be files, processes, or network connections. Process events record operations on processes, e.g., fork or execute. File events record operations on files, such as write, read, and rename. Network events record network access operations, such as socket send and socket recv.

2.2. Provenance graph

Provenance graphs (Lv et al., 2022; Liu and Jiang, 2023; Khosravi and Ladani, 2020) is a data structure extracted from audit logs that indicates the causal relationships between subjects and objects. Nodes (i.e., entities) represent subjects and objects in the target system, and edges indicate event relationships between subjects and objects. Each entity is characterized by its own semantics, and entity semantics (Ren et al., 2022; Luo et al., 2024) involves a comprehensive understanding of the properties and features of various system entities. For example, the semantics of a file may include its type, name, and permissions. The semantics of a process may include its origin, execution path, and permissions. Event relationships are represented by directed edges that include timestamps and event names, and this information helps analysts understand the sequence of events and causal relationships in the temporal dimension.

2.3. Causality analysis

Causal analysis (Hassan et al., 2019; Kwon et al., 2018; Zhu et al., 2023; Hossain et al., 2017; Kim et al., 2010; Goel et al., 2005) plays a key role in syslog analysis tasks, as it digs deeply into log information to infer dependencies, which helps security professionals understand and track the causal relationships related to POI events. For example, consider two events e1(u1,v1) and e2(u2,v2), where u1, u2, v1, and v2 represent nodes and ts(e) and te(e) denote the start time and the end time of event e, respectively. When v1 = u2 and ts(e1) < te(e2), a causal relationship is considered to exist between e1 and e2. The causal analysis process can be divided into two parts (King and Chen, 2003; King et al., 2005): (1) a backward causal analysis, which is used to identify the entry point of an attack, and (2) a forward causal analysis, which is used to investigate the consequences of the attack. Given a POI event es(u,v), the backward causal analysis traces from the source node u to find all events that have causal dependencies on u, whereas the forward causal analysis traces from the sink node v to find all events that v has a causal dependencies on.

2.4. ATT&CK model

The MITRE ATT&CK (MITRE ATTCK, 2024) model is a globally recognized knowledge base that compiles the tactics and techniques used by adversaries in cyberattacks on the basis of real-world security incidents and adversarial behaviors. The purpose of the ATT&CK model is to help security professionals identify and understand the behavior patterns of attackers, enabling more effective defenses and responses to various cybersecurity threats. The ATT&CK model is structured into multiple levels, primarily consisting of two core components: tactics and techniques. Tactics refer to the goals or objectives that attackers aim to achieve during their attack processes. Each tactic represents an intention or behavior of the attacker, such as initial compromising, privilege escalation, lateral movement, etc. Techniques, on the other hand, describe the specific methods and tools that attackers use to achieve these tactics, such as initial infiltration through phishing emails or gaining remote control via malware. The ATT&CK model enables security professionals to identify and understand attacker behavior patterns, allowing them to better defend against and respond to a wide range of cybersecurity threats.

3. Overview

Fig. 1 shows the architecture of LinTracer. Given an alarm point, LinTracerautomatically identifies the attack paths contained in the dependency graphs generated via a causal analysis and analyzes the source, scope, and consequences of the attack. LinTracerconsists of three modules: (1) a data collection and compression module, (2) an attack tracing module, and (3) an attack analysis and judgment module.

Threat Model. Our threat model is similar to that of previously developed system monitoring methods. We assume that the kernel and kernel-level auditing framework are parts of our trusted computing base (TCB) and that existing software and kernel hardening techniques can be used to protect the log store. Kernel-level attacks that intentionally compromise the security auditing system are outside the scope of this paper. We assume that an external attacker attacks the system from a remote location (outside the system). Thus, the attacker either exploits a system vulnerability or convinces the user to download a file with a malicious load.

We do not consider attacks using implicit streams (e.g., side channels) or interprocess communications (IPCs) that are not audited at the kernel level and thus cannot be captured by the underlying source tracker. Fine-grained auditing tools that capture memory traces or program analysis techniques can be used to address these types of attacks, but that is not the focus of this paper. We also do not consider mimicry attacks, where an attacker deliberately evades an intrusion detection system by acting on an intermediate point in a seemingly benign chain of events in an organization. The existing intrusion detection systems often rely on heuristics or analyses based on the attributes of individual events, making them vulnerable to such attacks. Although the current detection systems have limitations in terms of detecting mimicry attacks, addressing these shortcomings is beyond the scope of this paper, as our focus is on identifying contextual information about the events that are associated with the alerts generated by detection systems.

4. Design

In this section, we present the design details of each phase shown in Fig. 1. Specifically, Section 4.1 describes how LinTracer collects and compresses system audit logs, Section 4.2 describes how LinTracer computes the weight of each edge in the constructed dependency graph to generate a suspicious tracing graph, and Section 4.3 describes how LinTracer analyzes and judges the attack path based on the basis of the suspicious tracing graph.

4.1. Data collection and compression

In this section, we show how LinTracer leverages system monitoring to collect auditing logs of system activities; data compression is applied to the collected logs to simplify the constructed traceability graph.

4.1.1. Data collection

Among the available log collection techniques (Bryan Cantrill and Gregg, 2017; Redhat, 2023; Sysdig, 2017; Pohly et al., 2012), the Scalable Performance Analytics Data Engine (Gehani and Tariq, 2012) (SPADE) is an operating system-level data collector that automatically collects, filters, and stores system logs to comprehensively monitor and record a wide range of system activities. The SPADE uses node data structures and event data structures to record key system activity information. Node data structures mainly record various system entities, such as processes, files, and sockets, whereas event data structures describe the operational relationships between these entities. The process node data structure usually includes information such as the process number, parent process number, executed commands and permissions. Among these components, the parent process number can record the parent process of a process, whereas the commands and permissions executed by the process can be used to determine whether dangerous operations are involved in the process. The file node data structure mainly includes information such as the file path name, which is used to identify sensitive files or potentially threatening files contained in the system. Structures designed for network sockets usually include information such as local addresses, local ports, remote addresses, remote ports, socket types and host identifiers. The event data structure plays a key role in the log analysis process and typically contains the type of an event, the subject that executed the event, and the object that was executed. The data captured by the SPADE, which are subject-object-event triples, are fed into LinTracer in temporal order.

4.1.2. Data compression

Data compression reduces the resource consumption level and prevents cache pressure, but the compressed data must maintain the dependencies of the original data and must be considered to maintain the integrity of the traceability results. We propose semantic-based compression and temporary file deletion techniques based on two insights to simplify the provenance graph. (1) If the semantics of the source entity have not changed and the destination entity has received multiple identical information streams (events) from the source, these events share the same semantics and can be combined to eliminate redundant data. (2) Temporary files are typically generated by operating systems and applications during their runtimes to store intermediate

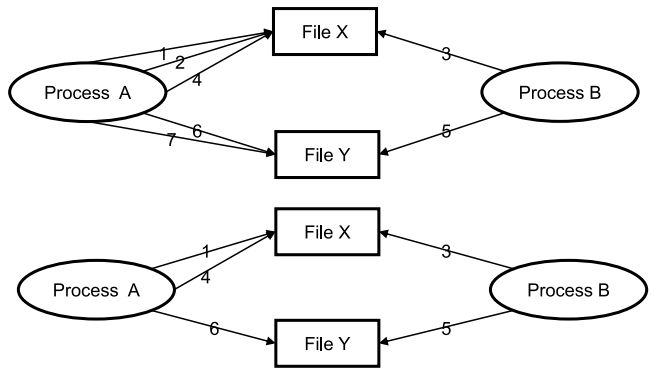

Fig. 2. An example of the redundant semantic deletion process.

data, cached information, or temporary states. These files are generally not needed for long-term storage and should be promptly deleted after a task is completed. Since temporary files typically do not contain critical information that is directly related to the attack, deleting them does not impact the integrity of the attack chain.

(1) Semantic-based compression. This method maintains a semantic table for recording the latest semantic information of each node, which is composed of <subject, event>, where an event is composed of <subject, event type, object>. When the semantic information carried by an incoming event already exists in the up-to-date semantic table, it can be determined that the event is redundant; thus, it can be deleted. This strategy aims to perform data compression without affecting the accuracy and completeness of the results. In 6, the relationships between two processes and two files are shown; the processes are connected by seven edges, each of which represents a specific event, and the numbers on the edges indicate the order in which these events occurred. For file X, at time point 1, when the first event occurs, process A performs a read operation on file X, which allows process A to acquire semantic information about file X. Prior to time point 2, the semantics of file X have not changed; therefore, when time point 2 is reached, process A again performs a read operation on file X. This does not introduce new semantic information to process A; therefore, the event at time point 2 can be removed. However, if the event at time point 2 is not a read operation but rather a write operation, it is not removable because this would result in a modification of the semantics of file X. At time point 3, process B performs a write operation on file X, which changes the semantics of file X. Therefore, at time point 4, when process A reads file X again, file X already has new semantic information, and process A introduces new semantics from process B. Therefore, the event at time point 4 cannot be deleted (see Fig. 2).

(2) Temporary file deletion. During attack investigations, temporary files usually have small attack impacts, so their deletion can effectively reduce the complexity of the original graph and improve the efficiency of the attack investigation process. We delete these temporary files by identifying the correlations between file creation events and deletion events and combining file attributes and behavioral features for judgment purposes. If no semantic changes are exhibited by other entities between the creation and deletion events of a temporary file, this can be regarded as a temporary file that does not affect the traceability analysis and can be processed for deletion.

4.2. Attack tracing

In this section, how LinTracer computes the weights of the edges via four types of features, including the size of the data flow, the time span of the events, the frequencies of the events, and the access degrees of the nodes, is shown. Then, LinTracer identifies suspicious attack paths by constructing suspicious tracing graphs and utilizing entity semantic score propagation algorithms.

4.2.1. Feature scoring computation

Feature extraction. For each edge, we select the size of the data flow, the time span of the events, the frequencies of the events, and the access degrees of the nodes as features. First, the size of the data flow can be compared with that of the data flow carried by the alert event to determine the anomalous degree of the data flow. If the size of the data flow is closer to that of the alert event, this data flow is more likely to be abnormal. Second, analyzing the time spans between event occurrences can help determine the activity pattern and attack window of the attacker. The closer the time of the event is to the time of the alarm message, the more likely the event is to be anomalous. Third, the event frequency analysis process requires attention to be paid to the frequencies of anomalous events. When determining event frequencies, high-frequency events are not necessarily abnormal, as certain normal system operations may lead to the frequent occurrence of events. In contrast, the sudden appearance of some rarely occurring events is more likely to be abnormal. Finally, the in- and out-degrees of a node reflect the criticality of the node in the dependency graph. Nodes with high in-degrees are usually critical nodes in the attack chain, and their exposure may have a significant effect on the system. For example, if a process has an abnormally high degree of incidence, it may be an attack node that deserves further investigation. For the features selected above, feature quantization is also required to convert the event features into computable and comparable values.

To determine the data flow size, we record the amount of data carried by each event. Then, the size of the data carried by each event is compared with the size of the data carried by the events directly related to the POI; this step is calculated according to Eq. (1) for the data flow characteristics. $S_{flow(e)}$ represents the importance of the data flow of edge e(u,v) to the POI event. Here, e represents the current event, $DataSize_e$ denotes the size of the data flow carried by the current event, and $DataSize_{POI}$ denotes the size of the data flow carried by the POI event. We introduce a positive number α to address the case in which both data sizes are equal and $S_{flow(e)}$ reaches the maximum value: $1/\alpha$.

$$S_{flow(e)} = \frac{1}{(|DataSize_e - DataSize_{POI}| + \alpha)} \tag{1}$$

To determine the time span of an event, we record the timestamp of each log during the analysis process. The timestamp of each event is then compared with the timestamps of events directly that are related to the POI. Since the timestamps captured by the SPADE are UNIX timestamps, they also need to be converted (Eq. (2)) and then processed according to the time span feature calculation formula (Eq. (3)). In the formulas, e represents the current event, $TimeGap_e$ refers to the time span between the timestamp of the current event e and the timestamp of the POI event, and $MaxTimeGap$ refers to the maximum number of time spans. $Timestamp_{POI}$ and $Timestamp_e$ refer to the converted values of the POI event and the timestamps of event e, respectively. We design $S_{time(e)}$ to model the criticality of the time span of an edge e(u,v) to the POI event:

$$S_{time(e)} = 1 - \frac{TimeGap_e}{MaxTimeGap} \tag{2}$$

$$TimeGap_e = |timestamp_{POI} - timestamp_e| \tag{3}$$

To determine the event frequency, we use the event frequency information derived from the event type frequency database. First, we obtain the maximum and minimum frequency scores for each event type in the database. Then, for each event type, its relative score is calculated on the basis of the maximum and minimum frequencies, as shown in Eq. (4). $Freq_e$ refers to the proportion of event type e among all event types, $Freq_{min}$ refers to the probability of the event type with the smallest occurrence frequency, and $Freq_{max}$ refers to the probability of the event type with the largest occurrence frequency. In this work, we assume that an event type with a lower occurrence

frequency is more suspicious. We map the data to the range [0, 1] to make the frequency scores of different events more comparable. We design $S_{freq(e)}$ to model the frequency criticality of an edge e(u,v) to the POI event:

$$S_{freq(e)} = 1 - \frac{(Freq_e - Freq_{min})}{(Freq_{max} - Freq_{min})} \tag{4}$$

To determine the access degree of a node, we obtain the subject and object information of each event and save it in the information of each node. If a node v has only one incoming edge from the source node, this indicates that the dependency of the node is highly concentrated on v. Therefore, the in/out degree features of the node can be determined via Eq. (5). $Out(v)$ refers to the out-degree of node v in event $e(u, v)$, and $In(v)$ refers to the in-degree of node v. We design $S_{deg(e)}$ to model the criticality of the access degree of a node for an edge e(u,v) to the POI event:

$$S_{deg(e)} = \frac{Out(v)}{In(v)} \tag{5}$$

After calculating the different features, the magnitudes of the feature values may vary greatly. To ensure that the different features have the same scale, we normalize them (Eq. (6)). $feat$ refers to one of the four eigenvalues, $Feat_{min}$ refers to the smallest value of this feature among all the edges, and $Feat_{max}$ represents the maximum value of this feature.

$$NorFeature(feat) = \frac{(feat - Feat_{min})}{(Feat_{max} - Feat_{min})} \tag{6}$$

Clustering. We introduce the notion of critical edges, which typically exhibit a different set of attributes and are more relevant to POI events in terms of those attributes. For example, a critical edge that involves reading data from a suspicious IP and then writing the data to a malicious script file may have data traffic that is similar to the size of the script file. After we determine the features of the edges, we use the K-SUMS algorithm to further mine the intrinsic structure of the edges and classify them into different clusters. We choose K=2 since we want to cluster the edges into two groups. This decision is made on the basis of the specific needs of the tracing task and the advantages of the K-SUMS algorithm, which is efficient enough to quickly and accurately handle massive amounts of data and has demonstrated good adaptability in terms of addressing multidimensional features.

Feature calculation. When faced with a large amount of data and many multidimensional features, classifying events into critical and noncritical edges with only simple clustering methods may not allow the potential relationships and differences between the data values to be explored. To further increase the differentiation of edges and the resulting classification accuracy, we adopt the linear discriminant analysis (LDA) method for feature projection purposes. LDA is a classic dimensionality reduction technique whose goal is to find the optimal projection vector ϵ while maintaining minimal differences within sample classes and maximal differences between sample classes. In tracing research, massive amounts of log data are collected when the number of events is large, and LDA can effectively extract key features from high-dimensional data and project the data to a low-dimensional space for better classification and analysis. By maximizing the Fisher criterion, i.e., maximizing the interclass dispersion and minimizing the intraclass dispersion, LDA is able to find the optimal projection vectors, thus effectively delineating the critical and noncritical edges.

On the basis of the extracted features, we calculate the weight values of the edges ($W(e)$) with the following formula after projecting the features:

$$W(e) = \epsilon 1 * S_{flow(e)} + \epsilon 2 * S_{time(e)} + \epsilon 3 * S_{freq(e)} + \epsilon 4 * S_{deg(e)} \tag{7}$$

where $\epsilon 1$, $\epsilon 2$, $\epsilon 3$, and $\epsilon 4$ are the corresponding values in the optimal projection vector obtained from the feature projection ϵ. In addition, the four feature values mentioned in Eq. (7) are the feature values that are processed after the normalization stage.

In contrast with the simple feature averaging method, our chosen feature projection scheme retains as much group differentiation information as possible and enables better attack entrance ordering for the subsequent implementation. To incorporate the entity semantic weight score propagation algorithm proposed in the subsequent sections, it needs to be ensured that each of the outgoing edges of each node has a weight within the range of [0,1] and that the sum of the weights is equal to 1.0. In this study, the edge weights are normalized as shown in Eq. (8).

$$NorWeight(e) = \frac{W(e)}{\sum_{e' \in out(u)} W(e')} \tag{8}$$

where u refers to the subject u of edge $e(u, v)$ and $out(u)$ refers to all outgoing edges of subject u.

4.2.2. Attack path extraction

Suspicious tracing graph construction. The suspicious tracing graph aims to reveal the sources and propagation paths of potential threat behaviors and security events. The suspicious tracing graph is constructed as follows: first, starting from the POI nodes, the backward causal analysis method is used to identify all the nodes that are related to these POI nodes. These nodes are subsequently processed in an iterative manner until attack entrances are found. In this study, attack entrances are defined as nodes that have not been accessed. This process is shown in Algorithm 1.

Algorithm 1 Construction of a suspicious tracing graph

Input: POI nodes
Output: Suspicious tracing graph, G

```
Initialization: G ← ∅, nodeSet ← ∅
nodeSet ← nodeSet ∪ {node}
for each u in node.vedList do
    if u in nodeSet then
        Continue
    end if
    nodeSet ← nodeSet ∪ {u}
    G ← G ∪ {(u, v, findEdge(u, v))}
    Use u to recursively continue the algorithm
end for
return G
```

Entity semantic score propagation. After a suspicious tracing graph is generated through backward causal analysis, multiple potential attack entry points are often identified. However, not all of these attack entry points are real security threats, so further screening and validation steps are needed to improve the accuracy and credibility of the traceability analysis results. After constructing the suspicious event tracing graph, we employ a series of techniques to further process and analyze it. First, we execute feature score calculations to quantify the edges contained in the suspicious tracing graph. By calculating the feature value and weight of each edge, we quantify the importance of this edge and its influence on the attack path to more accurately identify potential attack entrances. Then, we use our proposed entity semantic score propagation algorithm, which further filters the attack entrances with higher attack relevance values by semantically analyzing and propagating the entity nodes in the suspicious tracing graph. The algorithm combines the semantic correlation and propagation rules of entities, and this method can more accurately identify real attack sources and provide stronger support for security threat tracing and analysis tasks.

In Algorithm 2, the weight score of each POI node is first initialized to 1, and the difference in each cycle is updated in real time by setting a cursum and a presum. Then, any node u in the suspicious tracing graph is traversed to find the child nodes of node u, and the weight score of node u is updated in real time by the algorithm. The value of the gap

and the value of each node are updated every time a cycle is completed, and when the condition for jumping out of the cyclic process is met, the algorithm ends and calculates the weight scores for all nodes to obtain a suspicious tracing graph with weights. The propagation procedure terminates when the aggregate difference between the current iteration and the previous iteration is smaller than the gap value, indicating that the scores of all nodes have reached a stable point. We set the gap to 1e–3, as this yields value robust results according to our evaluations.

Algorithm 2 Entity semantic score propagation

Input: Suspicious tracing graph, G
Output: Suspicious tracing graph with weight, G_{weight}

```
POI.score ← 1
presum ← 0
cursum ← 0
while gap > 0.001 do
  gap ← 0
  for each u in G do
    if u = POI then
      continue
    else
      childNodeSet ← findNodeVList(u)
      diff ← 0
      for each v in childNodeSet do
        diff ← v.score × NorWeight(e(u, v)) + diff
        u.score ← diff
        cursum ← cursum + diff
      end for
    end if
  end for
  gap ← |cursum − presum|
  presum ← cursum
  cursum ← 0
end while
return G_weight
```

Suspicious attack path extraction. After implementing screening via the entity semantic score propagation algorithm, we extract the attack entrances that are most relevant to the attacks. Following this further analysis, we select the top ten attack entrances in terms of their entity semantic scores across multiple types for a detailed examination. These types include file types, process types, and socket types. Notably, these file types refer to files that are read by a certain process, and in this paper, these file types are the entry points of the attack, not the process. In addition, socket types are mainly involved with network connections to external entities. These socket types are included in the study because they often carry critical information regarding interactions with external attackers. After the initial ranking step, we perform further filtering by extracting the entity semantic scores of different types of attack entrances.

$$\beta = \frac{\sum_{i \in allNode} Score_i}{sumOfNode} \tag{9}$$

In Eq. (9), allNode refers to all attack entrances of the same type, Score represents the entity semantic scores of the attack entrances, and sumOfNode refers to the number of attack entrances of that type. The threshold β represents the average of the top ten attack entrances ranked according to the entity semantic scores produced for each type. If the entity semantic score of an attack entry is higher than this threshold, the entry is considered more likely to be related to an attack. The main purpose of this process is to improve the accuracy of the attack entrance screening process by determining the importance of each attack entrance from the weighted average of the associated entity semantic scores. Through this method, researchers can more accurately identify potential attack entrances and further examine their correlations with attack behaviors.

After further filtering the attack entry points, the paths between the attack entry points that exceed the threshold and the POI nodes, which are called the suspicious attack paths for this tracing result, are extracted. Obtaining these paths helps prepare for the subsequent analysis of the tracing results.

4.3. Attack judgment

In this section, we define labels and label passing rules based on ATT&CK. LinTracer calculates the semantic scores of the attack path after performing label propagation. Finally, LinTracer selects the attack path.

4.3.1. Label definition and event selection

Process labels. We classify process labels into two main categories, namely, state labels and operation labels, as shown in Table 2. The state labels are used to pass semantic information from the parent process to the child process via the system to create child process events. The operation labels are used to describe the operations performed by the process to pinpoint threats and are not passed to the child process through a system FORK event. For example, if a process accesses a high-value data stream node, it is labeled LPS2. If a process executes a sensitive command, it is labeled LPF2.

File labels. We classify file labels into two categories, namely, untrusted labels and high-value labels, as shown in Table 3. An untrustworthy file is one that contains data from the network; such files are usually generated during an attack process rather than initially existing in the system. High-value files are those that contain sensitive data, which are often the first files to be stolen and accessed by attackers in APT attacks. For example, if a file is generated because it is uploaded by a user, it is labeled LFR1 because the internal data of a file uploaded by a user are not always secure, and the existence of the file may lead to subsequent attack actions such as loading and executing attack code.

Socket labels. Network sockets transmit data and communications, which directly affects the security and stability of the examined system. With a blacklist-whitelist policy, we set three different labels for sockets, as shown in Table 4.

Highly suspicious labels. The label definitions above do not cover all the labels; we list only some labels that may appear during experiments, and not all of these labels indicate dangerous behaviors. On this basis, we define some labels that may indicate higher attack occurrence probabilities as highly suspicious labels (HSLs). Some of the highly suspicious labels used are shown in Table 5. In the future, with the continuous evolution of attack techniques and the emergence of new threat scenarios, the HSL list will continue to be updated to adapt to the changing security threat environment.

Event selection. After the labels of the different system entities are defined, it is necessary to specify the valid events between the various entities for obtaining contextual semantics and transferring and analyzing labels. The events involved are a subset of the events recorded in the Linux kernel data collected by Auditd. These events include operations such as executing and cloning for processing, writing and reading for files, and sending and receiving for sockets. A detailed list of these events is shown in Table 6. During the actual attack tracing process, these events provide analysts with key clues and information that help reveal the full picture of the attack path and behavioral patterns.

Label passing rules. To obtain the semantic information of the APT attack chain and efficiently transfer it to the target entities, we propose label transfer rules based on the ATT&CK model, the details of which are detailed in Table 7. In these rules, the event type represents the class of events to which the current label transfer rule applies, the subject and object denote the two entity nodes involved in the current event, and the label "receptor" refers to the receiver during the label

Table 2
Process label definition.

Labels	Description	Category
LPS	Process status label	Status
LPS1	A process has an external network connection	
LPS2	A processes has accessed stream nodes that contain high-value data	
LPS3	A processes uses data from the network	
LPS4	A process has opened a file downloaded from the network	
LPS5	A Processes interacts with non-existent files	
LPS6	A process reads a file containing user information	
LPS7	A process reads a file that records log-related information	
LPF	An operations label	Operation
LPF1	A process executes a file from the network	
LPF2	A process executes sensitive file	
LPF3	A process executes sensitive commands	
LPF4	A process executes a command	
LPF5	A process executes a shell command	
LPF6	A process modifies the timing file	
LPF7	A process reads high-value information files	
LPF8	A process sends data to a network connection	
LPF9	A process receives data from a network connection	
LPF10	A process modifies a permissions file	

Table 3
File label definition.

Labels	Description	Category
LFR	Untrusted file labels	Untrusted
LFR1	The file was created by a user upload	
LFR2	The information in the file comes from web data	
LFR3	The file does not exist	
LFR4	The file was created during a webshell attack	
LFR5	The file was created during a RAT attack	
LFR6	The file was created during a fileless attack	
LFV	High-Value file labels	High-value
LFV1	The file contains data that can control timed tasks	
LFV2	The file contains data that can control user permissions	
LFV3	The file contains sensitive user information	
LFV4	The file keeps a history of the commands that have been executed	
LFV5	The file was created by a process that reads high value files	

Table 4
Socket label definition.

Labels	Description
LSD	Socket labels
LSD1	The socket is on the whitelist, indicating safety
LSD2	The socket is on the blacklist, indicating danger
LSD3	The socket is not on the blacklist or whitelist; such sockets are suspicious

Table 5
Highly suspicious labels.

Labels	Description
LPS2	The process accesses stream nodes that contain high-value data
LPS3	The process contains data from the network
LPF2	The process acts on sensitive documents
LPF3	The process executes sensitive commands

Table 6
System events.

Attribute	Type	Description
Process	FORK	Creating child processes
	CLONE	Sharing files that create child processes
	EXECUTE	Process execution
	LOADLIB	Process loading of external libraries
File	WRITE	Writing a file
	CREATE	Creating a file
	UPDATE	Modifying file permissions or attributes
	READ	Reading a file
Socket	SENDTO	Sending data
	RECVFROM	Receiving data

transfer step. For example, in the first rule, if a process (with the label LPS1) writes a file, the file will be tagged with "LFR2". This is because the label "LPS1" indicates that the process carries a network socket connection, and after the process writes the file, the file also carries the semantics of the network socket connection. In Fig. 3, we provide an example of label passing.

4.3.2. Attack path label propagation

We propose a label passing method based on forward causal analysis. This method takes the suspicious entrances in the attack path as starting points, and through a careful analysis of each node, it determines whether the node exhibits behaviors related to the defined labels and then passes the corresponding labels to the relevant entities according to the label passing rules. This process involves not only checking the current node but also passing new labels according to the rules to ensure that the semantic information contained in the attack path is effectively delivered.

First, we conduct a detailed analysis of each node in the attack path and determine whether the node needs to be tagged with the corresponding label on the basis of its behavioral characteristics. These labels reflect the key attributes of the node, such as the types, states, and behaviors of processes, files, and sockets. Additionally, we consider the defined label passing rules. If the behavior of the current node satisfies the conditions of the label passing rules, the new label is passed to the relevant entity accordingly. This means that during the label passing process, the previous node is checked to determine whether it meets the conditions for triggering label passing, and the new label

Table 7
Transfer rules.

Event type	Subject	Object	Tagged receptor	Description
Write	Process	File	File	The process writes a file, which will be labeled with the file corresponding to the label carried by the process.
Write	Process	File	Process	A process modifies a file that controls user permissions or a timed task, and the process will be tagged with the process label corresponding to the label carried by the file.
Read execute	Process	File	Process	A process reads or loads the contents of a file, and the process is labeled with the process label corresponding to the label carried by the file.
Sendto	Process	Socket	Socket	The process sends data to the network, and the socket is labeled with the HSL label of the process.
Recvfrom	Process	Socket	Process	The process receives data from the network, and the process is labeled with the label carried by the socket.

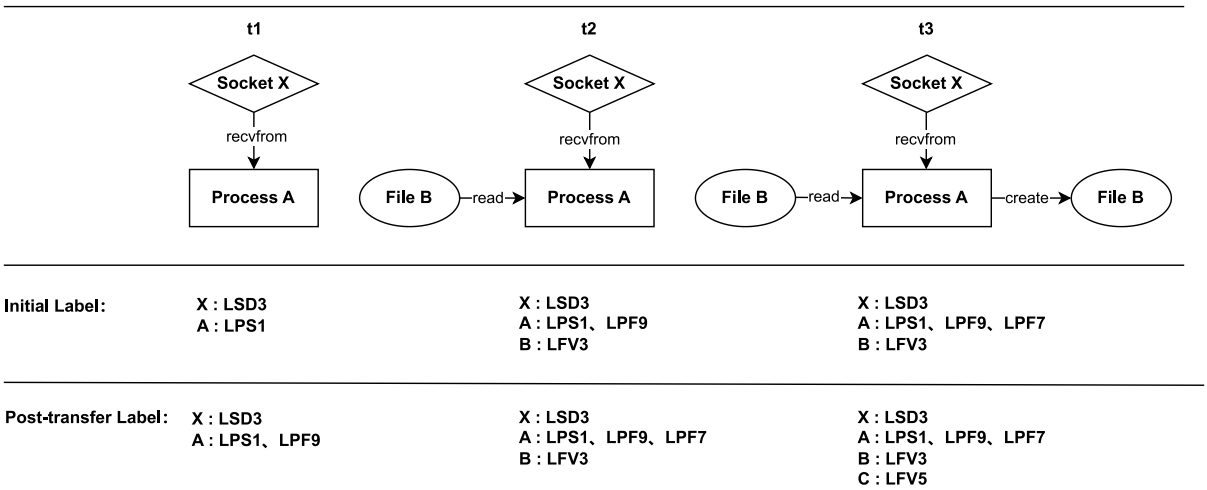

Fig. 3. Label transfer rule example.

is passed to the relevant entity according to the set rules to effectively pass the semantic information along the attack path.

4.3.3. Attack path analysis and judgment

Path score computation. When APT attack tracing and analysis are involved, the label score is a key factor to consider. In this paper, we propose a path-level label score calculation method based on label importance and relevance values. First, we assign a comprehensive and quantifiable score to each tag to help determine its importance and value for each label on the attack path. Then, a relevance score is assigned to the combination of labels by extracting the labels on the path after applying the label passing rule. Ultimately, by setting the labels and calculating the semantic scores of the labels on the paths according to the label passing rules, the attack paths can be better studied to determine the key attack entities and behaviors. Specifically, the path labeling score formula proposed in this paper is as follows:

$$S_{PathLabel} = \sum_{label \in allNode} label * ImpWeight + Rescore \tag{10}$$

In this formula, "allNode" refers to all the nodes on the path, and "label" refers to the labels carried by these nodes. In addition, the importance weight ImpWeight is a parameter that measures the importance of a label for identifying and tracing the attack process. This weight is determined on the basis of the criticality of the label and how strongly it affects the attack activity. Some of the tag importance weights are shown in Table 8. The relevance score "Rescore" is used to assess the degrees of association between labels, and its computation depends on the relationships between labels. In this paper, the relevance scores are calculated with the help of the ATT&CK model for tag combinations, which more accurately reflects the associations between labels. Similarly, the partial relevance scores are designed as shown in Table 9.

Table 8
Importance weights.

Label	ImpWeight
LFR1, LSD1	0.1
LPS1, LPF4, LPF9, LFR2, LFR3, LSD3	0.2
LPS4, LPS6, LPS7, LPF5, LPF6, LPF7, LPF8	0.3
LPS2, LPS3, LPS5, LPF1, LPF2, LPF3	0.4

Table 9
Relevance scores.

Label set	Rescore
LPS4&LPF4 | LPF1&LPF5 | LPS5&LPF5	0.5
LPF4 & (LPF6 | LPS6 | LPS7)	0.5
(LPS4&LPF5 | LPF1&LPF5 | LPS5 & LPF5) & LPF7	1
LPS7 | LPF1 & (LPS2 | LPF3) & LPS1	1
LPF1 | LPS5 & LPF5 & (LPF6 | LPS6 | LPS7) &LPF7	1

To comprehensively consider the behaviors and correlations of each node on an attack path, we add temporal features to the path computation process via the label semantics, as specified in Eq. (11). $S_{PathLabel}$ is obtained via a path-level label semantic computation for that path, whereas $S_{TimeDiff}$ is obtained from the time interval between the attack

Table 10
Details of our datasets.

Dataset	Duration (h)	Platform	Source	Attack description
S-1	24	Ubuntu16.04	Laboratory	Webshell attack
S-2	24	Ubuntu16.04	Laboratory	Remote access trojan from a phishing website
S-3	24	Ubuntu16.04	Laboratory	Living-off-the-land attack from a vulnerable service
D-1	168	Ubuntu14.04	DARPA Engagement	Information gathering and exfiltration & Malicious file downloading and execution
D-2	168	Ubuntu14.04	DARPA Engagement	In-memory attack on firefox

entrance and the POI nodes of the path, as specified in Eq. (12). Specifically, as shown in Eq. (12), MaxTimeDiff and MinTimeDiff refer to the maximum and minimum time differences of all the identified paths, respectively, and TimeDiff refers to the time difference between the attack entry and POI node of the current path. In this work, we extract path-level event characteristics by comparing the timestamps of the POI events and attack entrance events. When the difference between two timestamps is larger, this indicates greater potential and persistence for the attack; therefore, a higher score is assigned.

$$PathScore = S_{PathLabel} + S_{TimeDiff} \tag{11}$$

$$S_{TimeDiff} = \frac{MaxTimeDiff - TimeDiff}{MaxTimeDIff - MinTimeDiff} \tag{12}$$

Attack path selection. After computing the semantic scores on the basis of path-level labeling, we rank the trace graphs generated by different entry nodes based on the path-level scores; the trace graphs with higher scores are more likely to contain attack paths. Finally, the trace graphs generated from different types of attack entry points are summarized and ranked, and the top-ranked trace graphs are selected in each category.

5. Evaluation

5.1. Experimental setup

Among the datasets used in our experiments, one dataset was collected from our laboratory, where we designed attack scenarios based on current, widely observed APT attack techniques. Log collection was performed from January 15 to January 21, 2024. The other dataset is derived from DARPA Engagement 4, which was conducted from November 8 to November 21, 2018 (it is not publicly available). The data from our laboratory were collected in a controlled environment via the SPADE on the Ubuntu 16.04 operating system (32 GB if memory), and datasets S-1, S-2, and S-3 were generated by simulating three real APT attack scenarios. Specifically, dataset S-1 represents a webshell attack, dataset S-2 is based on a remote access Trojan (RAT) attack, and dataset S-3 emulates a living-off-the-land attack. We implemented four attacks on real-world APT campaigns on the basis of the detailed reports of DARPA Engagement 4 (DARPA Transparent Computing, 0000) and generated datasets D-1 and D-2. Dataset D-1 contains three simulated attack scenarios: an hc attack, a cclean attack and a rootkit attack. A memory attack on the Firefox browser was simulated in dataset D-2 (see Table 10). The node information contained in each dataset is shown in Table 11. The statistical information of the dependency graphs generated from these seven attacks is shown in Table 12, where V denotes the number of nodes contained in the traceability graph after performing a causal analysis on the POI events and CE stands for the number of critical events (related to the attacks).

Evaluation metrics. In addition to measuring false positives (FP, detected edges that are not critical edges) and false negatives (FN, missing edges that are critical edges), we compute the false negative rate $FNR = FN/E_c$, where FN represents the number of false positives and E_c represents the number of critical edges (column CE in Table 12); we also compute the false positive rate $FPR = FP/E_{total}$, where FP represents the number of false positives and E_{total} represents the number of edges from the column "With Compression#E" in Table 15.

Table 11
Node information for each dataset.

Dataset	Number of processes	Number of files	Number of sockets
S-1	198	119	33
S-2	146	71	18
S-3	222	122	37
D-1	112 738	492 003	99 263
D-2	99 742	412 535	88 288

Table 12
Statistics of the dependency graph for multiple attacks.

Dataset	V	CE
S-1	33	34
S-2	28	29
S-3	29	24
D-1-hc	12	12
D-1-cclean	9	8
D-1-rootkit	11	12
D-2-firefox	9	10

5.2. Data compression experiment and analysis

We perform two optimization steps on the log data to reduce the size of the tracing graph. First, a redundancy semantic-based deletion strategy is adopted for all log events to reduce the amount of unnecessary duplicate data. Table 13 details the experimental results of this redundancy semantic-based deletion process. The results indicate that the total number of events contained in datasets S-1, S-2, and S-3 reaches 1 million, and after the redundancy semantic-based deletion step, nearly 35% of the redundant events are removed. In contrast, the number of event logs in datasets D-1 and D-2 from DARPA reaches 30 million on average, and the number of deleted events is approximately 18% of the total. The difference between the compression levels of the two datasets may be due to the fact that many instances of the same type of behavior, such as large numbers of read and write operations, are performed within a short period in the laboratory environment, whereas the DARPA dataset is closer to a real-world scenario. The redundancy semantic-based deletion strategy enables savings of at least 1344 ms per million data values. In contrast, the compression ratio achieved for the DARPA dataset is lower, but the time saved per million data values is greater.

To further reduce the number of nodes contained in the tracing graph, we delete temporary files. Table 14 details the experimental results of temporary file deletion, which show that the percentages of deleted temporary files are approximately 0.4% to 10%. The files deleted depend mainly on the degrees of interaction between the files

Table 13
Results of redundant semantics-based deletion.

Dataset	Total events	Skipped events	Saved time (ms)
S-1	1 139 862	380 185	1664 ms
S-2	1 068 147	356 024	1436 ms
S-3	1 002 610	333 342	1972 ms
D-1	27 978 043	5 034 617	4957 ms
D-2	33 743 425	5 878 141	6024 ms

Table 14
Results of temporary file deletion.

Dataset	Total files	Deleted temporary files
S-1	119	13
S-2	71	3
S-3	122	7
D-1	492 003	2624
D-2	494 717	3016

Table 15
Compression experiment comparison.

Dataset	Without compression		With compression	
	E	V	E	V
S-1	213	90	213	90
S-2	228	100	225	99
S-3	93	43	93	43
D-1-hc	23 369	6922	23 314	6910
D-1-cclean	15 956	6189	15 956	6189
D-1-rootkit	12 009	2393	12 005	2393
D-2-firefox	17 953	3625	17 950	3625

and processes in the dataset. The reason for the relatively high percentage of temporary files deleted from dataset S-1 is that this dataset contains many inactive temporary files.

Compressing the log data can improve the efficiency of tracing; however, the compressed data must maintain the dependencies and high accuracy of the original data so that important data information is not lost. To verify the impact of the log processing and compression methods adopted in this paper on the subsequent tracing operations, a before-and-after comparison experiment is conducted, and the results are shown in Table 15. E and V in the table represent the number of edges and the number of nodes contained in the suspicious tracing graph, respectively. From the data results, the size of the suspicious tracing graph generated after applying the log compression strategy does not change significantly. The subsequent work is carried out based on the generated suspicious tracing graph. It can be concluded that the compression strategy, while enhancing efficiency, does not adversely impact the subsequent work, nor does it result in the loss of significant semantics.

5.3. Attack tracing experiments and analysis

This study obtains significant results in terms of tracing attack paths, and the proposed method is able to effectively discover attack paths while maintaining a low false-alarm rate and a lack of attack leakages. Table 16 shows the experimental results produced under seven attack scenarios. E represents the number of edges on the attack paths traced back from the alarm points by the different methods. DEPIMPACT does not misidentify benign events as malicious in any of the seven attack scenarios; however, the provenance graphs generated by this method contain larger numbers of edges. In contrast, SLEUTH sometimes produces fewer edges in the provenance graphs obtained for these seven attack scenarios but may incorrectly report benign events as malicious. Compared with DEPIMPACT and SLEUTH, our approach demonstrates superior performance by generating fewer edges in its provenance graphs (the size of the tracing graph generated by LinTraceris at least 2× smaller than the second-best result), reducing the number of false positives to a minimum, and achieving zero misses.

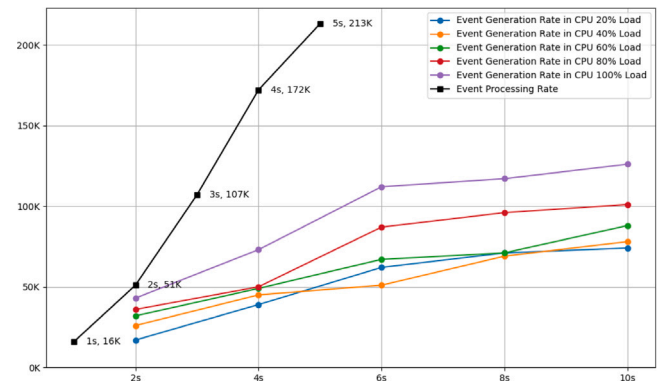

Fig. 4. Average event consumption rates and event generation rates under different CPU loads.

5.4. Real-time performance analysis

We test the real-time performance of the system by comparing the data processing rates and data generation rates achieved on the five datasets. In particular, the event generation rate is the number of logs collected in a given period, i.e., the number of logs collected per second, and is used to measure the rate of log data generation. The event consumption rate indicates the number of logs analyzed in the same amount of time and is used to evaluate the processing rate of the system. The acceleration ratio is the ratio between the event generation rate and the event consumption rate, which reflects the efficiency and real-time performance of the system in terms of processing data. As shown in Table 17, the event generation rates and event consumption rates vary greatly across the datasets; this is because the numbers of each event type are different in different datasets, and different types of events are processed in different ways. In all the datasets, the acceleration ratios are greater than 40, which indicates that the event consumption rate of the system is much higher than the event generation rate; i.e., the system is able to process the data at a fast speed and has good real-time performance.

We also test the average event generation rates of LinTracer under CPU loads of 20%, 40%, 60%, 80%, and 100% on the Ubuntu 16.04 operating system (32 GB of memory), as shown in Fig. 4. As the CPU load increases, the system is able to generate more data, as well as that collected by the SPADE. Moreover, we test the average event consumption rate of LinTracer. The results show that although more data are collected as the CPU load increases, the event consumption rate of LinTracer is greater than the event generation rate. When the CPU is fully loaded, the event consumption rate is 1.2 to 1.5 times the event generation rate. LinTracer maintains good performance under different CPU loads, and its data processing strategy effectively handles the increased data generation rate, demonstrating strong real-time processing capabilities.

5.5. Attack research and judgment analysis

The attack judgment experiment is based on the results of the attack tracing experiment, and it is designed to assess the tracing graph obtained in the tracing phase and further streamline the tracing graph by introducing information such as labels and path features to reduce the number of false positives. Table 18 shows the results of the research and judgment analysis; it indicates that our proposed APT attack research and judgment method is able to further refine an existing suspicious tracing graph and reduce the number of false positives. The addition of label propagation reduces the FPR by at least 10%, and the size of the attack graph generated by LinTraceris at least 3× smaller than that observed in the attack tracing phase.

Table 16
Tracing experiment results.

Dataset	SLEUTH			DEPIMPACT			LinTracer		
	FP	FN	E	FP	FN	E	FP	FN	E
S-1	76	5	105	87	0	121	75	0	109
S-2	83	4	108	83	0	112	65	0	94
S-3	33	7	50	38	0	62	30	0	54
D-1-hc	5756	6	5762	6100	0	6112	2647	0	2659
D-1-cclean	6433	5	6436	6914	0	6922	2790	0	2798
D-1-rootkit	4042	3	4051	3716	0	3728	969	0	981
D-2-firefox	2471	2	2479	4951	0	4961	1237	0	1247
Avg FP/FN/E	2699	4.57	2713	3127	0	3145	1116	0	1134
Avg FPR/FNR	0.35	0.31	/	0.39	0	/	0.23	0	/

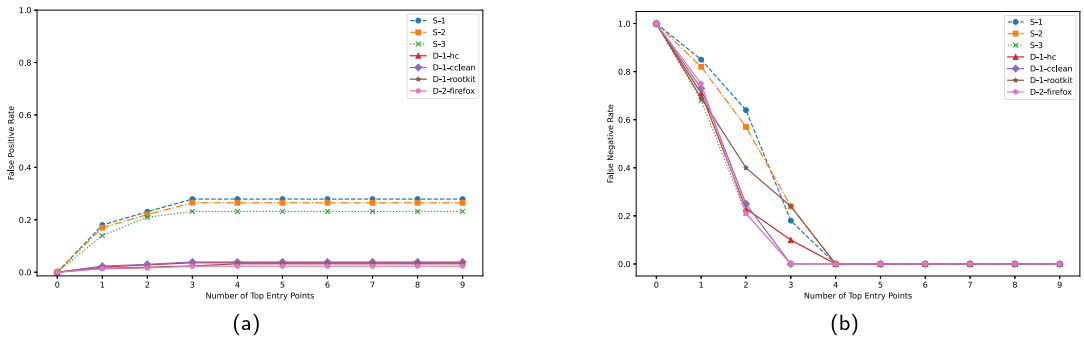

Fig. 5. Number of top entry points. (a) False positive rate. (b) False negative rate.

Table 17
Real-time performance.

Dataset	Event generation rate	Event consumption rate	Acceleration ratio
S-1	906.7301587	112 186.0209	123.7259176
S-2	1273.979762	106 432.1163	83.54301968
S-3	289.2537356	89 831.95602	310.5645492
D-1	961.4447766	69 424.42432	72.20843672
D-2	1166.785097	56 711.63866	48.60504202

Fig. 6 shows the attack tracing graph generated after conducting the research and judgment analysis for dataset S-3; it shows in detail the child processes forked by the large number of parent processes, as well as the associations between them. In practical attack scenarios, the operations of child processes are often regarded as the operations of their parent processes, which omits some attack events, thus simplifying the attack path. In contrast, our approach considers the associations between parent and child processes, files and sockets throughout the procedure and thus generates a more exhaustive trace path with a greater number of edges, which leads to more false positives than that produced for the simplified attack path. While the greater number of edges may increase the complexity of the analysis, it also provides more comprehensive information about the attack paths, which helps to reveal the attacker's behavioral patterns and attack strategies.

5.6. Suspicious path ranking analysis

To validate the effectiveness of selecting the top-ranked entry nodes to reveal attack sequences, we show how the increase in the number of selected attack entry nodes impacts the effectiveness of the method proposed in this study in terms of both the false-positive rate (FPR) and the false-negative rate (FNR). The FPR and FNR values in Figs. 5(a) and 5(b) show the impacts of different numbers of attack entry nodes on the results of the study. As more entrance nodes are used, the FNR gradually decreases, while the FPR increases. When the FNR value decreases to zero (when 4 to 5 nodes are used in different attack scenarios), the FPR value increases significantly if additional attack entry nodes continue to be selected for the final result. On the basis of this observation, we select the suspicious paths represented by the top 5 attack entry nodes of each type as the final result. This is because each additional attack entry node may add three new types of suspicious paths represented by the attack entrances, which significantly increases the complexity of the final attack graph.

5.7. System overhead analysis

The CPU and memory usage levels of DEPIMPACT and LinTracer for five different datasets are shown in Table 19. We use a performance monitor to record the average overhead of the system when it is running. In terms of CPU usage, for datasets S-1 to S-3 and D-1 to D-2, the CPU usage rates of the DEPIMPACT method are 13.4%, 14.0%, 13.8%, 24.2%, and 26.1%, whereas the CPU usage rates of LinTracer are 9.8%, 10.4%, 10.5%, 15.8%, and 16.7%, respectively. Compared with the DEPIMPACT method, the method proposed in this paper results in lower CPU usage levels on all datasets. In terms of memory usage, for datasets S-1 to S-3 and D-1 to D-2, the memory levels used by the DEPIMPACT method is 1385.5 MB, 1428.7 MB, 1452.5 MB, 3357.3 MB and 3657.1 MB, whereas those of LinTracer are 504.1 MB, 576.3 MB, 619.8 MB, 1846.5 MB and 2064.5 MB. 1846.5 MB and 2064.6 MB. A comparison among the experimental results produced on the five datasets reveals that our method has lower CPU utilization and lower memory overhead than does DEPIMPACT. The reason for this may be that the forward dependency graph used by the DEPIMPACT method has too many nodes and edges that overlap with the backward dependency graph, which requires more memory and a greater CPU utilization rate. In short, the results show that our method significantly reduces the memory overhead required on all datasets, indicating a significant advantage in terms of system performance.

6. Discussion

Despite the progress made by the system proposed in this paper in terms of addressing advanced persistent threat (APT) attacks, some areas for improvement remain.

(1) In terms of feature selection, the dimensions and types of features used can be further expanded and deepened. The current work

Table 18
Judgment results.

Dataset	DEPIMPACT			DEPIMPACT + Label passing			LinTracer		
	FP	FN	E	FP	FN	E	FP	FN	E
S-1	81	0	115	67	0	101	50	0	84
S-2	72	0	101	61	0	90	52	0	81
S-3	38	0	62	27	0	51	16	0	40
D-1-hc	5205	0	5217	4576	0	4588	838	0	850
D-1-cclean	5479	0	5487	4168	0	4176	618	0	626
D-1-rootkit	2951	0	2963	2381	0	2393	374	0	386
D-2-firefox	4265	0	4275	3615	0	3625	407	0	417
Avg FP/FN/E	2584	0	2603	2128	0	2146	336	0	355
Avg FPR/FNR	0.35	0	/	0.28	0	/	0.13	0	/

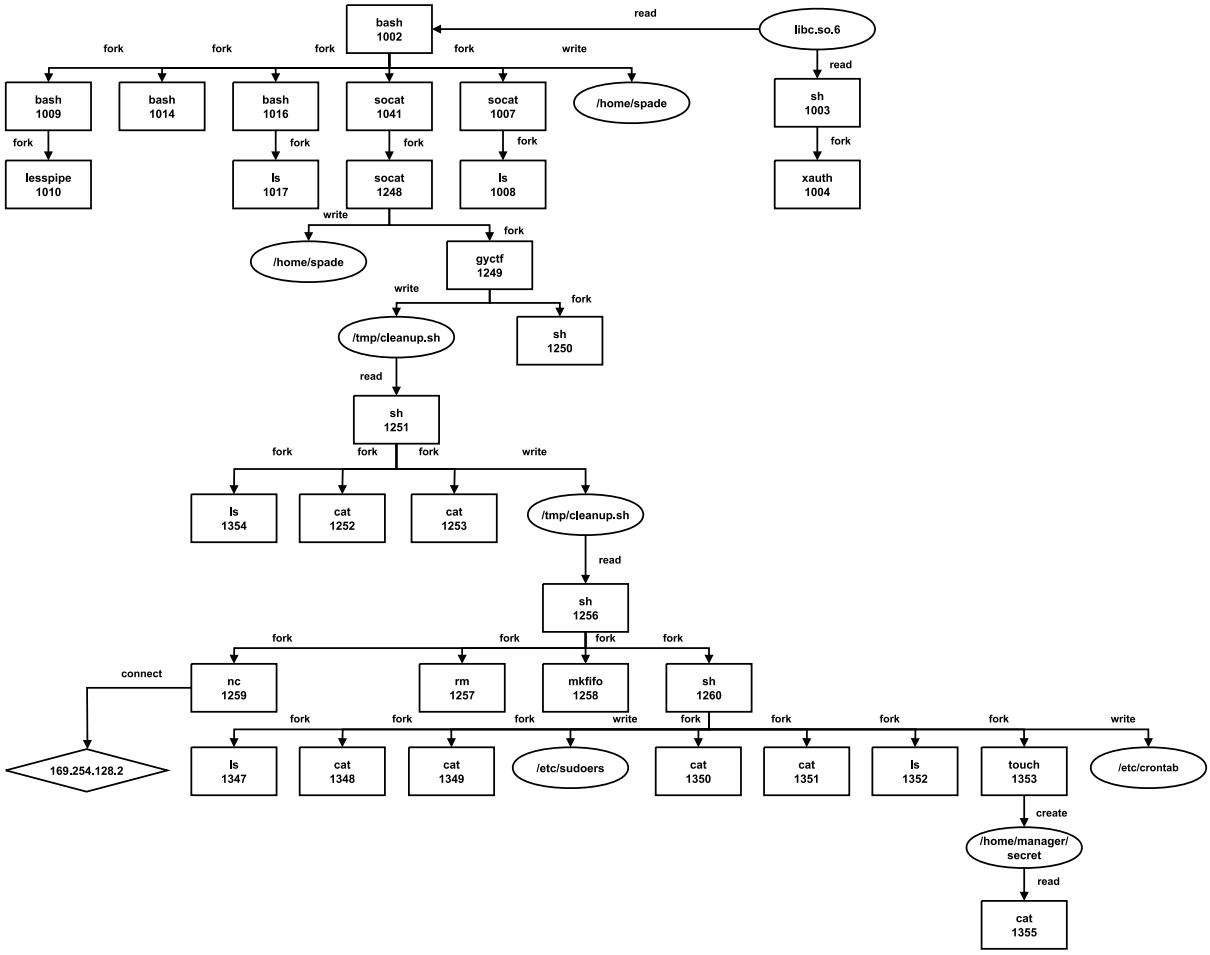

Fig. 6. Attack source tracing diagram of dataset S-3. An attacker detects that a particular service is running on an open port of the target host, which has a string overflow vulnerability, and the attacker exploits this to interact with the service and gain user privileges. By writing the corresponding script, the attacker successfully exploits the string overflow vulnerability to perform a fileless attack and finally obtains the user rights to run the service. The attacker performs read and write operations on sensitive files and user credentials and then transmits them to an external network.

Table 19
CPU and memory usage levels required for different datasets.

Dataset	DEPIMPACT		LinTracer	
	CPU (%)	Memory (MB)	CPU (%)	Memory (MB)
S-1	13.4	1385.5	9.8	504.1
S-2	14.0	1428.7	10.4	576.3
S-3	13.8	1452.5	10.5	619.8
D-1	24.2	3357.3	15.8	1846.5
D-2	26.1	3657.1	16.7	2064.6

focuses primarily on the global and local features of events, and future research could consider incorporating a wider array of features, such as the correlations, frequency distributions, and temporal relationships of events, to increase the ability of the system to recognize complex attack behaviors. For example, diverse features such as network traffic characteristics, host behaviors, and user behaviors could be integrated to obtain comprehensive analyses and portrayals of the behavioral patterns and features of cyberattacks.

(2) In terms of selecting and optimizing clustering algorithms, although this paper employs the K-SUMS clustering algorithm and a feature projection scheme based on linear discriminant analysis (LDA), potential remains for exploring clustering algorithms that are more suitable for complex data analysis tasks and for further improving the accuracy and efficiency of these algorithms. In particular, clustering algorithms that are designed for high-dimensional data and align

more closely with the feature extraction process for APT attack events should be considered. These algorithms should maintain their sensitivity and robustness to anomalous data while reducing the incurred computational complexity. Additionally, the real-time performance and response speed of the system require further optimization. It is possible to further improve the existing log reduction techniques and the time complexity of traceback methods and to employ parallel computing and distributed computing technologies to increase the processing capabilities and real-time responsiveness of the system. Moreover, concerning the issue of label contamination, future research could consider the introduction of a label reduction mechanism. This would involve the deletion of certain labels from nodes under specific conditions to mitigate the accumulation and contamination of labels, ensuring the stability and reliability of the system.

(3) In terms of data compression, LinTracer adopts a semantic-based compression method, specifically, redundancy semantic skipping. The insight behind this method is as follows. If the semantics of the source entity have not changed and the destination entity receives multiple identical information flows (events) from the source entity (which have the same semantics), these events can be merged to eliminate redundant data. For example, if process P continuously reads file A and the semantics of the subject and object remain unchanged, only the first READ event will be retained. However, if process Q writes to file A during this period, changing the semantics of file A, the semantics of the subsequent read of file A by process P will also change. Therefore, the next three events are retained: READ (process P reads file A), WRITE (process Q writes to file A), and READ (process P reads file A). Skipping redundant semantic data does not lead to the loss of contextual semantics or causality. The purpose of an attack investigation is to determine the cause and scope of the attack. Redundant events are removed, but events with the same semantics are retained. LinTracer can still identify the events that led to the attack without affecting the accuracy of the investigation. The data compression method in this paper only considers the single-host scenario. In real-world scenarios, if communication between different hosts can be matched where the semantics remain unchanged, the compression rate can be further improved to reduce tracing overhead. The main challenge of this issue arises from communication delays, which can be addressed by security analysts through synchronized time windows.

(4) In terms of attack judgment, although the system relies on a predefined APT taxonomy, its adaptability to continuously evolving or varied attacks is not limited. The entities included in the system primarily include processes, files, and sockets, and system events typically record the operations initiated by the subject on the object. LinTracer uses different tactical stages derived from the ATT&CK model to define labels for system entities and designs label transition rules to ultimately achieve label aggregation. Then, we assign relevance scores to label combinations to filter out attack paths. Through entity labels and transition rules, LinTracer can reflect the nature of the examined attack by tracking untrusted data flows, untrusted control flows, and high-value data flows. In practical deployment, given the freely configurable nature of labels, when new attack tactics emerge, security analysts can, based on their own experience, easily expand the corresponding labels with the help of knowledge bases such as Atomic Red Team (Atomic Red Team, 0000). This enables the system to adapt to new attack scenarios in a timely manner. APTSHIELD (Zhu et al., 2023) has successfully applied this method to attack detection.

(5) In terms of reducing false-positive rates, the traceback strategies and analysis methods of the system could be further refined. For example, by introducing more nuanced threshold adjustment methods and adaptive mechanisms or by establishing more precise labeling strategies as well as whitelists and blacklists, the rate of false positives could be further decreased, thereby improving the accuracy and credibility of the system.

7. Related work

Data compression. Audit frameworks are known to generate enormous log volumes, exceeding gigabytes per day on a single machine (Ma et al., 2018; Bates et al., 2015b). At times, this creates a "needle-in-a-haystack" problem for analysts, as they must sift through extraneous log data searching for real evidence. Numerous efforts have been made to reduce the burden imposed on security analysts by removing uninteresting and redundant information. Bates et al. (2015a, 2017) and LogGC (Lee et al., 2013) used entity pruning strategies to filter out benign system entities and remove isolated entities that did not interact with other entities. Winnower (Hassan et al., 2018) and NodeMerge (Tang et al., 2018) aggregated entities with the same function on the basis of heuristic rules or template techniques to reduce the number of entities. Hossain et al. (2018) proposed a redundant node optimization (RNO) strategy for achieving dependency-preserving reduction. When new entity information is generated, an entity is updated only if the entity did not previously have a large impact on another entity. Xu et al. (2016) proposed an event-merging algorithm that merges repeated events by identifying entities with the same forward and backward tracking capabilities. Zhu et al. (2021) proposed a lightweight global semantic preservation method that removes duplicate operations from system activities only when the semantics of the examined entity have not changed. KCAL (Ma et al., 2018) detects redundant events at the kernel cache level, thereby reducing the overhead associated with transferring, processing, and writing redundant raw logs. PrioTracker (Liu et al., 2018) and NoDoze (Hassan et al., 2019) use compression methods based on statistical analysis to remove data that are not relevant to the target attack. PrioTracker determines whether an event is anomalous on the basis of its occurrence frequency and topological characteristics and uses the hill climbing algorithm (Russell and Norvig, 2003) from the field of machine learning to make the obtained weight assignment results more reasonable. NoDoze considers anomalies in events to propagate and to aggregate outliers across the suspected dependency paths. Holmes (Milajerdi et al., 2019) removes spurious tactics, techniques and procedures (TTPs) on the basis of benign preconditions and data traffic analyses, employing the concept of ancestral coverage to discard low-confidence chains of suspicious events. RapSheet (Hassan et al., 2020a) removes entities that are unrelated to alert events, resulting in a minimal skeleton graph that retains only the causal relationships between alert events.

Attack detection. Recent studies have explored a variety of techniques that analyze provenance graphs for achieving automated host intrusion detection, with a particular focus on advanced persistent threats. Flash (Rehman et al., 2024) employed a Word2Vec-based semantic encoder to capture the key semantic attributes in provenance graphs and designed a lightweight classifier that combines graph neural networks (GNNs) and Word2Vec embeddings to discover potential threats. NODLINK (Li et al., 2024) is an online APT attack detection system that achieves high-precision and fine-grained attack detection under the constraints of limited resources and timeliness by modeling the APT attack detection process as a Steiner tree problem (STP) and utilizing efficient online approximation algorithms. However, this approach may not always guarantee an optimal solution when complex or nonlinear attack paths are encountered. KAIROS (Cheng et al., 2024) leverages a novel graph neural network-based encoder–decoder architecture that learns the temporal evolution process of the structural changes exhibited by a provenance graph to quantify the degree of anomalousness for each system event. R-CAID (Goyal et al., 2024) precomputes the root causes of each node during the graph construction process and then directly links those nodes to their root causes during the embedding step. Furthermore, the classification model of R-CAID is a node-/process-level model. MAGIC (Jia et al., 2024) is a self-supervised APT detection method that uses masked graph representation learning for conducting anomaly detection across different supervision levels and adapts to

concept drift. Rehman et al. (2024), Cheng et al. (2024), Jia et al. (2024) and Goyal et al. (2024) used anomaly based detection methods, which are often found to be unreliable under extremely changing conditions. The use of labels by LinTracer enables it to accurately identify the entities and behaviors associated with an attack and can reduce the rate of false positives. In contrast, anomaly-based detection methods may generate more false positives because of unusual patterns exhibited by normal behavior.

Attack investigation. When a security alert is triggered, a quick attack investigation can determine the cause and scope of the attack so that appropriate measures can be taken to minimize the impact of the attack. HERCULE (Pei et al., 2016) uses the Louvain method of community detection to discover attack communities in heterogeneous audit logs and then uses these attack communities as a basis for determining correlations among threat alerts. Zhai et al. (2006) were the first to use kernel-level audit logs to determine the correlations among alerts. They leveraged the Backtracker system (King and Chen, 2003) to generate dependency graphs from kernel-level audit logs and then used these graphs to correlate threat alerts. Similarly, Holmes (Milajerdi et al., 2019) and RapSheet (Hassan et al., 2020a) use data provenance to derive the correlations among threat alerts. In contrast, BotHunter leverages network-level communication to identify the stages of a botnet infection and correlate those stages. Moreover, CLARION (Chen et al., 2021) and ALASTOR (Datta et al., 2022) determine and investigate provenance in the cloud computing context. DEPIMPACT (Fang et al., 2022), NoDoze (Hassan et al., 2019) and Swift (Hassan et al., 2020b) prioritize generated alerts on the basis of anomalous contextual histories. DEPIMPACT identifies the critical component of a dependency graph of a POI event generated via causality analysis, which preserves attack information and filters out irrelevant dependencies. Unfortunately, NoDoze and Swift are applicable only if anomalous events are observed in the network. RapSheet (Hassan et al., 2020a) introduces the notion of tactical provenance graphs (TPGs) to determine the causal dependencies between EDR-generated threat alerts. Zhong et al. (2016) mined past analysts' security operation traces to learn alert triage rules and then used these rules to automate the alert triage process. OmegaLog (Hassan et al., 2020c) leverages a static binary analysis technique to parse and interpret application event logs and generates semantically aware and execution-partitioned provenance graphs. ALchemist (Yu et al., 2021) applies log normalization techniques and manually written Datalog rules to integrate application and system logs without any program instrumentation. In addition to application logs, UIScope (Yang et al., 2020) applies UI elements and events of GUI applications to help partition the low-level system logs generated during a long-running process. WATSON (Zeng et al., 2021) leverages contextual information in log-based knowledge graphs to enable semantic inference. To distinguish representative behaviors, WATSON provides a vector representation of behavior semantics and uses this vector to cluster semantically similar behaviors. ATLAS (Alsaheel et al., 2021) leverages a novel combination of causality analysis, natural language processing, and machine learning techniques to build a sequence-based model, which establishes the key patterns of attack and nonattack behaviors from a causal graph. AIRTAG (Ding et al., 2023) leverages unsupervised learning to train a DL model on log texts. During the investigation process, the DL model directly takes log files as inputs and predicts entities that are related to the attack.

8. Conclusion

We propose an efficient system for tracking APT attack chains that is with fused entity semantics and focused on Linux systems. Initially, the system employs strategies such as redundant semantic skipping and nonviable node pruning to filter massive amounts of log data, effectively reducing the computational overhead of the system. By extracting the global and local features of events and integrating the K-SUMS clustering algorithm with a feature projection scheme based on linear discriminant analysis (LDA)-based feature projection scheme, the system accurately then computes feature scores for each edge, thereby increasing the precision and accuracy of the attack event identification process. Furthermore, an event anomaly score propagation algorithm that is capable of rapidly and accurately tracing potential attack entry points is proposed in this paper, providing crucial clues for the subsequent analysis and judgment stages. By employing label and path anomaly score-based algorithms, the system further refines the characteristics of the target attack chain and improves the ability to trace back to the sources of attacks, offering an efficient and reliable analytical method for achieving cybersecurity in Linux systems. Our evaluation of the developed method on real-world attacks demonstrates the effectiveness of our research in terms of filtering out irrelevant dependencies while preserving those that are related to attacks.

CRediT authorship contribution statement

Tiantian Zhu: Writing – review & editing, Writing – original draft, Software, Resources, Methodology, Investigation, Funding acquisition. **Wenya He:** Writing – original draft, Software, Investigation, Data curation. **Tieming Chen:** Supervision, Resources, Project administration, Funding acquisition. **Jiabo Zhang:** Visualization, Methodology, Formal analysis. **Mingqi Lv:** Supervision, Resources, Project administration. **Hongmei Li:** Writing – review & editing, Writing – original draft, Investigation. **Aohan Zheng:** Methodology, Investigation, Data curation. **Jie Zheng:** Validation, Software, Data curation. **Mingjun Ma:** Writing – review & editing, Visualization, Data curation. **Xiangyang Zheng:** Visualization, Supervision, Resources. **Zhengqiu Weng:** Visualization, Resources, Formal analysis. **Shuying Wu:** Writing – review & editing, Validation, Supervision, Methodology, Funding acquisition.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work is supported in part by the following grants: National Natural Science Foundation of China under Grant No. U22B2028, 62372410, 62002324, and U1936215. Zhejiang Provincial Natural Science Foundation of China under Grant No. LQ21F020016 and LZ23F020011. Key R&D Projects in Zhejiang Province, China under Grant No. 2021C01117. Major Program of Natural Science Foundation of Zhejiang Province, China under Grant No. LD22F020002. The Fundamental Research Funds for the Provincial Universities of Zhejiang, China under Grant No. RF-A2023009. Wenzhou Key Scientific and Technological Projects, China under Grant No. ZG2024007. Wenzhou Basic Scientific Research Projects, China under Grant No. G2024033.

Data availability

Data will be made available on request.

References

Alsaheel, Abdulellah, Nan, Yuhong, Ma, Shiqing, Yu, Le, Walkup, Gregory, Celik, Z Berkay, Zhang, Xiangyu, Xu, Dongyan, 2021. {AtlAS}: A sequence-based learning approach for attack investigation. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 3005–3022.

Atomic Red Team, https://github.com/redcanaryco/atomic-red-team.

Bates, Adam, Butler, Kevin R.B., Moyer, Thomas, 2015a. Take only what you need: leveraging mandatory access control policy to reduce provenance storage costs. In: 7th USENIX Workshop on the Theory and Practice of Provenance. TaPP 15.

Bates, Adam, Tian, Dave Jing, Butler, Kevin RB, Moyer, Thomas, 2015b. Trustworthy {whole-system} provenance for the linux kernel. In: 24th USENIX Security Symposium (USENIX Security 15). pp. 319–334.

Bates, Adam, Tian, Dave, Hernandez, Grant, Moyer, Thomas, Butler, Kevin RB, Jaeger, Trent, 2017. Taming the costs of trustworthy provenance through policy reduction. ACM Trans. Internet Technol. (TOIT) 17 (4), 1–21.

Bryan Cantrill, Adam Leventhal, Gregg, Brendan, 2017. Dtrace. http://dtrace.org/.

Chen, Xutong, Irshad, Hassaan, Chen, Yan, Gehani, Ashish, Yegneswaran, Vinod, 2021. {ClariON}: Sound and clear provenance tracking for microservice deployments. In: 30th USENIX Security Symposium (USENIX Security 21). pp. 3989–4006.

Cheng, Zijun, Lv, Qiujian, Liang, Jinyuan, Wang, Yan, Sun, Degang, Pasquier, Thomas, Han, Xueyuan, 2024. KAIROS: practical intrusion detection and investigation using whole-system provenance. In: 2024 IEEE Symposium on Security and Privacy. SP, pp. 3533–3551.

DARPA Transparent Computing, https://www.darpa.mil/program/transparent-computing.

Datta, Pubali, Polinsky, Isaac, Inam, Muhammad Adil, Bates, Adam, Enck, William, 2022. {AlastOR}: Reconstructing the provenance of serverless intrusions. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 2443–2460.

Ding, Hailun, Zhai, Juan, Nan, Yuhong, Ma, Shiqing, 2023. {AirtAG}: Towards automated attack investigation by unsupervised learning with log texts. In: 32nd USENIX Security Symposium (USENIX Security 23). pp. 373–390.

Fang, Pengcheng, Gao, Peng, Liu, Changlin, Ayday, Erman, Jee, Kangkook, Wang, Ting, Ye, Yanfang Fanny, Liu, Zhuotao, Xiao, Xusheng, 2022. {Back-propagating} system dependency impact for attack investigation. In: 31st USENIX Security Symposium (USENIX Security 22). pp. 2461–2478.

Gao, Peng, Shao, Fei, Liu, Xiaoyuan, Xiao, Xusheng, Qin, Zheng, Xu, Fengyuan, Mittal, Prateek, Kulkarni, Sanjeev R, Song, Dawn, 2021. Enabling efficient cyber threat hunting with cyber threat intelligence. In: 2021 IEEE 37th International Conference on Data Engineering. ICDE, IEEE, pp. 193–204.

Gao, Peng, Xiao, Xusheng, Li, Ding, Li, Zhichun, Jee, Kangkook, Wu, Zhenyu, Kim, Chung Hwan, Kulkarni, Sanjeev R, Mittal, Prateek, 2018a. {SaQL}: A stream-based query system for {real-time} abnormal system behavior detection. In: 27th USENIX Security Symposium (USENIX Security 18). pp. 639–656.

Gao, Peng, Xiao, Xusheng, Li, Zhichun, Xu, Fengyuan, Kulkarni, Sanjeev R, Mittal, Prateek, 2018b. {AiQL}: Enabling efficient attack investigation from system monitoring data. In: 2018 USENIX Annual Technical Conference (USENIX ATC 18). pp. 113–126.

Gehani, Ashish, Tariq, Dawood, 2012. SPADE: Support for provenance auditing in distributed environments. In: ACM/IFIP/USENIX International Conference on Distributed Systems Platforms and Open Distributed Processing. Springer, pp. 101–120.

Goel, Ashvin, Po, Kenneth, Farhadi, Kamran, Li, Zheng, De Lara, Eyal, 2005. The taser intrusion recovery system. In: Proceedings of the Twentieth ACM Symposium on Operating Systems Principles. pp. 163–176.

Goyal, Akul, Wang, Gang, Bates, Adam, 2024. R-CAID: Embedding root cause analysis within provenance-based intrusion detection. In: 2024 IEEE Symposium on Security and Privacy. SP, pp. 257–257.

Hassan, Wajih Ul, Aguse, Lemay, Aguse, Nuraini, Bates, Adam, Moyer, Thomas, 2018. Towards scalable cluster auditing through grammatical inference over provenance graphs. In: Network and Distributed Systems Security Symposium.

Hassan, Wajih Ul, Bates, Adam, Marino, Daniel, 2020a. Tactical provenance analysis for endpoint detection and response systems. In: 2020 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1172–1189.

Hassan, Wajih Ul, Guo, Shengjian, Li, Ding, Chen, Zhengzhang, Jee, Kangkook, Li, Zhichun, Bates, Adam, 2019. Nodoze: Combatting threat alert fatigue with automated provenance triage. In: Network and Distributed Systems Security Symposium.

Hassan, Wajih Ul, Li, Ding, Jee, Kangkook, Yu, Xiao, Zou, Kexuan, Wang, Dawei, Chen, Zhengzhang, Li, Zhichun, Rhee, Junghwan, Gui, Jiaping, et al., 2020b. This is why we can't cache nice things: Lightning-fast threat hunting using suspicion-based hierarchical storage. In: Proceedings of the 36th Annual Computer Security Applications Conference. pp. 165–178.

Hassan, Wajih Ul, Noureddine, Mohammad Ali, Datta, Pubali, Bates, Adam, 2020c. OmegaLog: High-fidelity attack investigation via transparent multi-layer log analysis. In: Network and Distributed System Security Symposium.

Hossain, Md Nahid, Milajerdi, Sadegh M, Wang, Junao, Eshete, Birhanu, Gjomemo, Rigel, Sekar, R, Stoller, Scott, Venkatakrishnan, VN, 2017. {SleuTH}: Real-time attack scenario reconstruction from {coTS} audit data. In: 26th USENIX Security Symposium (USENIX Security 17). pp. 487–504.

Hossain, Md Nahid, Wang, Junao, Weisse, Ofir, Sekar, R, Genkin, Daniel, He, Boyuan, Stoller, Scott D, Fang, Gan, Piessens, Frank, Downing, Evan, et al., 2018. {Dependence-preserving} data compaction for scalable forensic analysis. In: 27th USENIX Security Symposium (USENIX Security 18). pp. 1723–1740.

Jia, Zian, Xiong, Yun, Nan, Yuhong, Zhang, Yao, Zhao, Jinjing, Wen, Mi, 2024. {MagIC}: Detecting advanced persistent threats via masked graph representation learning. In: 33rd USENIX Security Symposium (USENIX Security 24). pp. 5197–5214.

Jiang, Xuxian, Walters, AAron, Xu, Dongyan, Spafford, Eugene H, Buchholz, Florian, Wang, Yi-Min, 2006. Provenance-aware tracing ofworm break-in and contaminations: A process coloring approach. In: 26th IEEE International Conference on Distributed Computing Systems. ICDCS'06, IEEE, 38–38.

Khosravi, Mehran, Ladani, Behrouz Tork, 2020. Alerts correlation and causal analysis for APT based cyber attack detection. IEEE Access 8, 162642–162656.

Kim, Taesoo, Wang, Xi, Zeldovich, Nickolai, Kaashoek, M Frans, 2010. Intrusion recovery using selective re-execution. In: 9th USENIX Symposium on Operating Systems Design and Implementation. OSDI 10.

King, Samuel T., Chen, Peter M., 2003. Backtracking intrusions. In: Proceedings of the Nineteenth ACM Symposium on Operating Systems Principles. pp. 223–236.

King, Samuel T, Mao, Zhuoqing Morley, Lucchetti, Dominic G, Chen, Peter M, 2005. Enriching intrusion alerts through multi-host causality.. In: Ndss.

Kwon, Yonghwi, Kim, Dohyeong, Sumner, William Nick, Kim, Kyungtae, Saltaformaggio, Brendan, Zhang, Xiangyu, Xu, Dongyan, 2016. Ldx: Causality inference by lightweight dual execution. In: Proceedings of the Twenty-First International Conference on Architectural Support for Programming Languages and Operating Systems. pp. 503–515.

Kwon, Yonghwi, Wang, Fei, Wang, Weihang, Lee, Kyu Hyung, Lee, Wen-Chuan, Ma, Shiqing, Zhang, Xiangyu, Xu, Dongyan, Jha, Somesh, Ciocarlie, Gabriela, et al., 2018. MCI: Modeling-based causality inference in audit logging for attack investigation. In: Network and Distributed Systems Security (NDSS) Symposium.

Lee, Kyu Hyung, Zhang, Xiangyu, Xu, Dongyan, 2013. Loggc: garbage collecting audit log. In: Proceedings of the 2013 ACM SIGSAC Conference on Computer & Communications Security. pp. 1005–1016.

Li, Zitong, Cheng, Xiang, Sun, Lixiao, Zhang, Ji, Chen, Bing, 2021. A hierarchical approach for advanced persistent threat detection with attention-based graph neural networks. Secur. Commun. Netw. 2021, 1–14.

Li, Shaofei, Dong, Feng, Xiao, Xusheng, Wang, Haoyu, Shao, Fei, Chen, Jiedong, Guo, Yao, Chen, Xiangqun, Li, Ding, 2024. Nodlink: An online system for fine-grained apt attack detection and investigation.

Liu, Haitian, Jiang, Rong, 2023. A causal graph-based approach for APT predictive analytics. Electron. 12 (8), 1849.

Liu, Yushan, Zhang, Mu, Li, Ding, Jee, Kangkook, Li, Zhichun, Wu, Zhenyu, Rhee, Junghwan, Mittal, Prateek, 2018. Towards a timely causality analysis for enterprise security. In: NDSS.

Luo, Bin, Chen, Liangguo, Ruan, Shuhua, Luo, Yonggang, 2024. Detecting APT-exploited processes through semantic fusion and interaction prediction. Comput. Mater. Contin. 78 (2).

Lv, Yang, Qin, Shaona, Zhu, Zifeng, Yu, Zhuocheng, Li, Shudong, Han, Weihong, 2022. A review of provenance graph based APT attack detection: applications and developments. In: 2022 7th IEEE International Conference on Data Science in Cyberspace. DSC, IEEE, pp. 498–505.

Ma, Shiqing, Lee, Kyu Hyung, Kim, Chung Hwan, Rhee, Junghwan, Zhang, Xiangyu, Xu, Dongyan, 2015. Accurate, low cost and instrumentation-free security audit logging for windows. In: Proceedings of the 31st Annual Computer Security Applications Conference. pp. 401–410.

Ma, Shiqing, Zhai, Juan, Kwon, Yonghwi, Lee, Kyu Hyung, Zhang, Xiangyu, Ciocarlie, Gabriela, Gehani, Ashish, Yegneswaran, Vinod, Xu, Dongyan, Jha, Somesh, 2018. {Kernel-supported}{cost-effective} audit logging for causality tracking. In: 2018 USENIX Annual Technical Conference (USENIX ATC 18). pp. 241–254.

Ma, Shiqing, Zhang, Xiangyu, Xu, Dongyan, 2016. Protracer: Towards practical provenance tracing by alternating between logging and tainting. In: 23rd Annual Network and Distributed System Security Symposium. NDSS 2016, Internet Soc.

Milajerdi, Sadegh M, Gjomemo, Rigel, Eshete, Birhanu, Sekar, Ramachandran, Venkatakrishnan, VN, 2019. Holmes: real-time apt detection through correlation of suspicious information flows. In: 2019 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 1137–1152.

MITRE ATTCK, 2024. https://attack.mitre.org/.

Pei, Kexin, Gu, Zhongshu, Saltaformaggio, Brendan, Ma, Shiqing, Wang, Fei, Zhang, Zhiwei, Si, Luo, Zhang, Xiangyu, Xu, Dongyan, 2016. Hercule: Attack story reconstruction via community discovery on correlated log graph. In: Proceedings of the 32Nd Annual Conference on Computer Security Applications. pp. 583–595.

Pohly, Devin J, McLaughlin, Stephen, McDaniel, Patrick, Butler, Kevin, 2012. Hi-fi: collecting high-fidelity whole-system provenance. In: Proceedings of the 28th Annual Computer Security Applications Conference. pp. 259–268.

Redhat, 2023. The linux audit framework. https://github.com/linux-audit/.

Rehman, Mati Ur, Ahmadi, Hadi, Hassan, Wajih Ul, 2024. FLASH: A comprehensive approach to intrusion detection via provenance graph representation learning. In: 2024 IEEE Symposium on Security and Privacy. SP, pp. 139–139.

Ren, Yitong, Xiao, Yanjun, Zhou, Yinghai, Zhang, Zhiyong, Tian, Zhihong, 2022. CSKG4APT: A cybersecurity knowledge graph for advanced persistent threat organization attribution. IEEE Trans. Knowl. Data Eng..

Russell, Stuart, Norvig, Peter, 2003. Artificial intelligence: A modern approach.

Sysdig, 2017. Sysdig. https://sysdig.com/.

Tang, Yutao, Li, Ding, Li, Zhichun, Zhang, Mu, Jee, Kangkook, Xiao, Xusheng, Wu, Zhenyu, Rhee, Junghwan, Xu, Fengyuan, Li, Qun, 2018. Nodemerge: Template based efficient data reduction for big-data causality analysis. In: Proceedings of the 2018 ACM SIGSAC Conference on Computer and Communications Security. pp. 1324–1337.

Wang, Qi, Hassan, Wajih Ul, Li, Ding, Jee, Kangkook, Yu, Xiao, Zou, Kexuan, Rhee, Junghwan, Chen, Zhengzhang, Cheng, Wei, Gunter, Carl A, et al., 2020. You are what you do: Hunting stealthy malware via data provenance analysis.. In: NDSS.

Xiong, Chunlin, Zhu, Tiantian, Dong, Weihao, Ruan, Linqi, Yang, Runqing, Cheng, Yueqiang, Chen, Yan, Cheng, Shuai, Chen, Xutong, 2020. CONAN: A practical real-time APT detection system with high accuracy and efficiency. IEEE Trans. Dependable Secur. Comput. 19 (1), 551–565.

Xu, Zhiqiang, Fang, Pengcheng, Liu, Changlin, Xiao, Xusheng, Wen, Yu, Meng, Dan, 2022. Depcomm: Graph summarization on system audit logs for attack investigation. In: 2022 IEEE Symposium on Security and Privacy. SP, IEEE, pp. 540–557.

Xu, Zhang, Wu, Zhenyu, Li, Zhichun, Jee, Kangkook, Rhee, Junghwan, Xiao, Xusheng, Xu, Fengyuan, Wang, Haining, Jiang, Guofei, 2016. High fidelity data reduction for big data security dependency analyses. In: Proceedings of the 2016 ACM SIGSAC Conference on Computer and Communications Security. pp. 504–516.

Yang, Runqing, Ma, Shiqing, Xu, Haitao, Zhang, Xiangyu, Chen, Yan, 2020. UIScope: Accurate, instrumentation-free, and visible attack investigation for GUI applications. In: NDSS.

Yu, Le, Ma, Shiqing, Zhang, Zhuo, Tao, Guanhong, Zhang, Xiangyu, Xu, Dongyan, Urias, Vincent E, Lin, Han Wei, Ciocarlie, Gabriela F, Yegneswaran, Vinod, et al., 2021. ALchemist: Fusing application and audit logs for precise attack provenance without instrumentation. In: NDSS.

Zeng, Jun, Chua, Zheng Leong, Chen, Yinfang, Ji, Kaihang, Liang, Zhenkai, Mao, Jian, 2021. WATSON: Abstracting behaviors from audit logs via aggregation of contextual semantics. In: NDSS.

Zhai, Yan, Ning, Peng, Xu, Jun, 2006. Integrating IDS alert correlation and OS-level dependency tracking. In: Intelligence and Security Informatics: IEEE International Conference on Intelligence and Security Informatics, ISI 2006, San Diego, CA, USA, May 23-24, 2006. Proceedings 4. Springer, pp. 272–284.

Zhao, Jun, Yan, Qiben, Liu, Xudong, Li, Bo, Zuo, Guangsheng, 2020. Cyber threat intelligence modeling based on heterogeneous graph convolutional network. In: 23rd International Symposium on Research in Attacks, Intrusions and Defenses. RAID 2020, pp. 241–256.

Zhong, Chen, Yen, John, Liu, Peng, Erbacher, Robert F., 2016. Automate cybersecurity data triage by leveraging human analysts' cognitive process. In: 2016 IEEE 2nd International Conference on Big Data Security on Cloud (BigDataSecurity), IEEE International Conference on High Performance and Smart Computing (HPSC), and IEEE International Conference on Intelligent Data and Security. IDS, IEEE, pp. 357–363.

Zhu, Tiantian, Wang, Jiayu, Ruan, Linqi, Xiong, Chunlin, Yu, Jinkai, Li, Yaosheng, Chen, Yan, Lv, Mingqi, Chen, Tieming, 2021. General, efficient, and real-time data compaction strategy for apt forensic analysis. IEEE Trans. Inf. Forensics Secur. 16, 3312–3325.

Zhu, Tiantian, Yu, Jinkai, Xiong, Chunlin, Cheng, Wenrui, Yuan, Qixuan, Ying, Jie, Chen, Tieming, Zhang, Jiabo, Lv, Mingqi, Chen, Yan, et al., 2023. Aptshield: A stable, efficient and real-time apt detection system for linux hosts. IEEE Trans. Dependable Secur. Comput..

Tiantian Zhu received the Ph.D. degree in computer science from Zhejiang University, Hangzhou, China, in 2019. He is currently an associate professor with the college of computer science and technology, Zhejiang University of Technology, China. His research interests include mobile security, system security and artificial intelligence.

Wenya He received the B.E. degree in computer science from the Zhejiang A&F University, Hangzhou, China, in 2021. She is currently pursuing the M.S. degree for computer science in Zhejiang University of Technology. Her current research interests and directions include APT attack investigation.

Tieming Chen received the Ph.D. degree in computer software and theory from BeiHang University, Beijing, China, in 2011. He is now a professor with the college of computer science and technology, Zhejiang University of Technology, Hangzhou, China. He is also a member of IEEE and ACM. His research interests include cyberspace security and intelligence security.

Jiabo Zhang received the M.S. degree in computer science from Zhejiang University of Technology, Hangzhou, China, in 2024. His research interests include Cyberspace security, Apt attack investigation.

Mingqi Lv received the Ph.D. degree in computer science from Zhejiang University, Hangzhou, China, in 2012. His research interests include spatiotemporal data mining and ubiquitous computing.

Hongmei Li received the B.E. degree in computer science from the Henan University of Traditional Chinese Medicine, Zhengzhou, China in 2023. Her is currently pursuing the M.S. degree for computer science in Zhejiang University of Technology. Her current research interests and directions include cyber threat detection.

Aohan Zheng received the B.E. degree in software engineering from Zhejiang University of Technology, Zhejiang, China, in 2022. She is currently studying for a master's degree in computer science at Zhejiang University of technology. Her current research interests and directions include Cyberspace Security , APT attack detection and privacy protection.

Jie Zheng received the B.E. degree in Internet of Things Engineering from Jiangxi University of Applied Science, Jiangxi, China, in 2023. He is currently studying for a master's degree in software engineering at Zhejiang University of technology. He current research interests and directions include Cyberspace Security , APT attack detection and malicious command line detection.

MingJun Ma obtained his B.E. degree in Computer Science from Zhejiang Shuren University, Hangzhou, China, in 2023. He is currently pursuing an M.S. degree in Computer Science at Zhejiang University of Technology. His current research interests focus on Advanced Persistent Threats (APT) and mobile security.

Xiangyang Zheng obtained a Master's degree in Software Engineering from Xiamen University, China, in 2016. He is currently a lecturer at the School of Data Science and Artificial Intelligence, Wenzhou University of Technology, China. His research interests include mathematical modeling, data analysis, and artificial intelligence.

Zhengqiu Weng received the Ph.D. degree in computer science and technology from the Zhejiang University of Technology, Zhejiang, China, in 2023, and the M.S. degree in software engineering from the Beijing Institute of Technology, Beijing, China, in 2005. She is also a professor with the Wenzhou University of Technology, Zhejiang, China. Her current research interests include networks security and big data technologies.

Shuying Wu received the M.S. degree in computer science and technology from Zhejiang University, Hangzhou, P.R. China, in 2006. She is now a associate professor with the Wenzhou Polytechnic, Wenzhou, P.R. China. Her current research interests include artificial intelligence and big data technologies.